

Evaluating the Effect of Generalization over Tabular Q-Learning methods

Franco Faundez¹

¹ Department of Economics, Management and Quantitative Methods, Università degli Studi di Milano, Via Conservatorio, 7, Milano, Italy
`francoantonio.faundez@studenti.unimi.it` .

Keywords: Reinforcement Learning, Q-Learning, Linear Function Approximation

1 Introduction

In Reinforcement Learning (RL) there are many model free methods used, for which one of the most widely known is Q-learning, which estimates the value of a state-action pair directly from observed transitions. When the state space is sufficiently small, it is common to store these values explicitly in a table, providing a tabular representation of the Q function.

The effectiveness of a tabular representation depends strongly on how the state of an environment is defined, and as information to describe it increases, the possible state-action pairs grow increasingly. This is particularly problematic in environments where we want the agent to generalize behavior rather than memorizing state values.

This project goes through the limitation of tabular methods in a partially observable grid navigation problem, where an agent must navigate from a fixed position to the goal region while avoiding traps and obstacles. Furthermore, each training episode is randomized, so rather than memorizing a single trajectory the agent is forced to generalize based on current local information.

Studying this effect is done through progressively increasing the complexity of both the environment and the learning method, starting from standard tabular Q-learning to the more complex linear function approximation (LFA) implementation. This project will also cover how feature engineering is crucial for the performance of LFA.

Finally, the main objective of this project is to investigate the tradeoff between explicit tabular representations and feature based value function approximation (VFA), particularly at generalization capabilities when exposed to different map configurations. In this sense, the analysis will be take into consideration feature design, training stability and computational requirements for execution of each method.

2 Problem Definition

This project considers a grid navigation environment in which an agent must learn to navigate from an initial position to a predefined goal region. Aside from free movement tiles, the environment also contains obstacles, which restrict the agent's movement in particular directions, and traps that penalize with a large negative reward when stepped on. The agent must therefore learn to reach the goal while avoiding penalties and finding viable routes to traverse different map layouts as effectively as possible.

A key distinction to a traditional MDP is that in this problem the agent has only partial observability of its surroundings. The map itself is hidden by fog of war, meaning that the agent does not have access to the complete configuration of the environment and instead, at each time step, it receives a local observation containing information about the tiles within its vicinity. As the agent explores the environment, this local view changes and new areas become observable.

Furthermore, the environment layout (or map) is randomly generated for each training episode, making it impossible for the agent to solve the task by memorizing a single optimal path or a fixed configuration. Therefore, it must learn a navigation strategy that can be applied to different map layouts based on information available within its local view.

With this setting in mind, traditional tabular Q-learning may result in the observation space to grow substantially depending on how observations are represented. This proves to be a useful setting to analyze the limitations of tabular reinforcement learning versus value function approximation implementations. In a tabular approach, action values are stored independently for each represented observation, and as the number of these representations increase, the dimensions of state-action values $Q(s,a)$ to be learned can also grow substantially.

In contrast, value function approximation represents action values using a set of features and learned parameters. Rather than keeping an independent value for every possible case, observations that share similar features can influence the same parameters. This allows learned information to generalize observations that have not been encountered but share features with from previously visited observations.

The primary objective is therefore to compare traditional tabular Q-learning with a value function approximation variant in a partially observable environment. Specifically, the project investigates how both approaches behave as the size of the state space increases and whether function approximation allows the agent to generalize more effectively than its tabular counterpart across an unseen map layouts.

Therefore, the research question addressed in this project is:

Does linear function approximation generalize more effectively than tabular Q-learning to unseen map configurations as the size of the environment increases?

3 Methodology

The methodology is divided into two main components: the design of the environment and the definition of learning methods used for training. For the final method, which uses LFA, an additional section is included to describe the feature engineering process, including the initial feature representations and the subsequent modifications that led to the final implementation. Most of the methods are based on course lecture notes [1] and Sutton and Barto [2].

3.1 Modeling the environment

We first define the base map layout that serves as a baseline environment for the experiments. As the methods and their observation representations are progressively modified, any changes with respect to this baseline definition will be stated accordingly.

Let H and W denote the height and width of the map, respectively. Each tile, or cell, is identified by its row and column coordinates,

$$r \in \{0, \dots, H - 1\}, \quad c \in \{0, \dots, W - 1\}.$$

For the standard tabular Q-learning implementation, the state is defined solely by the agent's current position. Each position is encoded as a single integer according to

$$s = rW + c.$$

The corresponding state space is therefore

$$\mathcal{S} = \{0, 1, \dots, HW - 1\},$$

with

$$|\mathcal{S}| = HW.$$

This initial representation is based on the microbe grid world example covered in class [1]. For the linear function approximation model, the tabular representation is replaced by a feature based implementation, which will be covered in the corresponding section.

The map contains different types of tiles. Let $\mathcal{S}_{\text{trap}} \subset \mathcal{S}$ denote the set of trap tiles and $\mathcal{S}_{\text{obstacle}} \subset \mathcal{S}$ the set of obstacle tiles. Additionally, s_{goal} denotes the goal state and s_{start} the initial state. All remaining states are considered as free tiles, where the agent can move without any restrictions.

By convention, the agent always starts in the upper left corner of the map. Consequently,

$$s_{\text{start}} = 0,$$

and the initial state distribution is deterministic:

$$p_0(s) = \begin{cases} 1, & s = s_{\text{start}}, \\ 0, & \text{otherwise.} \end{cases}$$

The action space consists of four discrete actions corresponding to the four cardinal directions:

$$\mathcal{A} = \{0, 1, 2, 3\} = \{\text{Up, Right, Down, Left}\}.$$

Each action $a \in \mathcal{A}$ corresponds to an attempted displacement of one tile in its respective direction. Before considering map boundaries or obstacles, the intended position after executing an action can be expressed as

$$(r', c') = (r, c) + \Delta(a),$$

where

$$\Delta(a) = \begin{cases} (-1, 0), & a = \text{Up}, \\ (0, 1), & a = \text{Right}, \\ (1, 0), & a = \text{Down}, \\ (0, -1), & a = \text{Left}. \end{cases}$$

The environment uses deterministic transition dynamics, meaning that, given a state s and an action a , the resulting state s' is uniquely determined by the current map configuration. If the intended movement is valid, the agent moves to the corresponding adjacent tile. If the movement is blocked by an obstacle or the map boundary, the agent remains in its current position.

Thus, for the resulting state $s' = f(s, a)$,

$$P(s' \mid s, a) = 1,$$

while

$$P(\tilde{s} \mid s, a) = 0 \quad \forall \tilde{s} \neq s'.$$

During training, an ϵ -greedy policy may cause the agent to select an exploratory action instead of its current greedy action. However, once an action has been selected, its outcome within the environment remains deterministic.

The reward received by the agent depends on the outcome of the selected action and the type of tile involved. The corresponding rewards are summarized in Table 1. The goal is the only terminal state. Entering a trap produces a large negative reward, but does not terminate the episode, so the agent remains able to select a new action from the trap tile on the following time step. Obstacles, in contrast, prevent movement into the intended tile. When the agent attempts to move into an obstacle, it receives a smaller negative reward compared to a trap and remains in its current state.

Initially, the task is treated as undiscounted, using a discount factor of

$$\gamma = 1.$$

The effect of modifying the discount factor is considered in subsequent experiments when evaluating the learning and convergence behavior of the different agents.

Landed tile type	Reward
Trap	-15
Obstacle	-5
Goal	+100
Otherwise	-1

Table 1 Rewards based on the type of tile landed on

The default configuration is set to $H = 12$ and $W = 12$, with s_{goal} located at row 12 and column 12. For the initial experiment, a fixed map layout containing traps and obstacles is used, as shown in Figure 1.

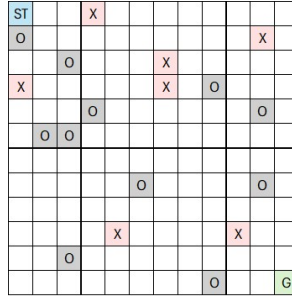


Fig. 1 Initial fixed layout. ST is defined as the initial starting tile, G is the goal tile, while the X's and O's are trap and obstacle tiles, respectively.

3.2 Learning methods

Given the previously defined environment, we now have sufficient information to introduce the first learning method. We begin with a basic model operating on the fixed map layout defined in the previous section. This serves as a baseline before introducing more complex observation representations and randomized map layouts. Ultimately, the objective is to evaluate how well different models generalize when the environment configuration changes.

a. Tabular Q-Learning

The first method is based on standard tabular Q-learning. The objective of Q-learning is to estimate the optimal action-value function, which represents the expected return obtained by taking an action a in state s and subsequently following the optimal policy.

An independent value is stored for every state-action pair in a Q-table, hence the action-value function is represented as

$$Q(s, a),$$

where $Q(s, a)$ estimates the value of selecting action a while in state s .

At each time step, the agent observes its current state s , selects an action a , receives a reward r , and transitions to the next state s' . The corresponding Q-value is then updated using the update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a' \in \mathcal{A}} Q(s', a') - Q(s, a) \right],$$

where $\alpha \in (0, 1]$ denotes the learning rate and $\gamma \in [0, 1]$ is the discount factor.

During training, actions are selected according to an ϵ -greedy behavior policy. With probability $1 - \epsilon$, the agent selects an action with the highest currently estimated Q-value, while with probability ϵ , it selects an exploratory action. The exploration probability ϵ is gradually decreased until reaching a defined minimum value. This allows the agent to explore the environment extensively during the early stages of training while increasingly exploiting its learned action values as training progresses.

b. Indexed Tabular Q-Learning

While standard tabular Q-learning is suitable for fixed map layout, it does not generalize to environments in which the positions of obstacles and traps vary between episodes. Hence, map randomization during training is introduced, for which state representation is augmented with information from the agent's local view.

In this setting, a new map layout is generated at the beginning of each episode, meaning that the distribution and traps is controlled by densities d_{trap} and d_{obstacle} , respectively, which determines the proportion of available grid tiles assigned for each trap and obstacle types. Then, a randomized placement is done to distribute these tiles inside the map, generating a new randomized layout. Finally this is followed by Breadth-First Search (BFS) algorithm to check if there's at least one viable path from the start to goal tile.

The local view is defined as any tile that the agent can see from his current position within 2-tiles of distance.

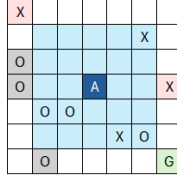


Fig. 2 Grid representation of the local view: tiles that are in blue are the ones the agent sees, while the others are under fog-of-war

Consequently, the state can no longer be represented solely by the agent's position or by a single integer index. Instead, in this "indexed" representation, each state is defined by the combination of the agent's current coordinates and its local observation:

$$s_t = (x_t, y_t, \ell_t),$$

where x_t and y_t denote the agent's position at time step (t), and ℓ_t represents an encoding of the corresponding local view. This tuple forms a state representation and can therefore be used as a key in a dictionary or an index. The action values are consequently stored as

$$Q((x_t, y_t, \ell_t), a),$$

rather than in a fixed Q-table indexed by a single discrete state identifier, as done in the previous method. Apart from this change in state representation and storage, the learning procedure remains identical to standard tabular Q-learning. In particular, the action values are updated according to

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a' \in \mathcal{A}} Q(s', a') - Q(s, a) \right]$$

with the future value term omitted for terminal transitions, and actions are selected using the same ϵ -greedy behavior policy described previously.

It will be demonstrated that this representation makes a substantial increase in the effective state space, since each distinct combination of position and local observation constitutes a different index. Hence, two states that are in structure very similar may be treated as completely independent states because experience obtained in one state does not directly update value estimates of the other. The update itself would be highly unlikely to happen just because of the state's space size.

c. LFA Q-Learning

To address the limitations of previously presented tabular methods, linear function approximation is introduced for estimating the action-value function. Instead of independently storing a value for every possible state representation, the Q-function is approximated from a set of features describing the state. This is highly useful for the problem since actions taken would depend more on the local view than the actual position of the agent. This allows information learned from one state to generalize to other states with similar feature representations.

For each action $a \in \mathcal{A}$, we define a weight vector $\mathbf{w}_a$. Given a state s , represented by a feature vector $\phi(s)$, its action value is approximated as

$$\hat{Q}(s, a; \mathbf{w}_a) = \mathbf{w}_a^\top \phi(s) = \sum_{i=1}^d w_{a,i} \phi_i(s),$$

where d is the number of features, $\phi_i(s)$ denotes the value of feature i in state s , and $w_{a,i}$ is the corresponding learned weight for action a .

As in tabular Q-learning, learning is based on the temporal difference (TD) error. For a non terminal transition (s, a, r, s') , the TD target is

$$y = r + \gamma \max_{a' \in \mathcal{A}} \hat{Q}(s', a'; \mathbf{w}_{a'}),$$

and the corresponding TD error is

$$\delta = y - \hat{Q}(s, a; \mathbf{w}_a).$$

An initial weight of 0 is set to all actions, and the algorithm learns the Q-function indirectly through the weights of each action.

A difference from the last method is that states sharing similar feature representations will have updates that also affect the estimated action values for both. This provides a better mechanism at generalization across different configurations of the local view. Features are primarily constructed from information contained within the agent’s local view. In this case, position is not considered since we want the agent to rely only on the current view and not memorize positions.

The feature set considered for this method is explained in the next section.

3.3 Feature Engineering

VFA depends strongly on the representation used to describe the state. The objective of feature engineering is to extract information that captures relevant properties of the agent’s surroundings while allowing generalization over different map layouts [3].

In this section, the feature vector is built from information available only within the agent’s local view. These features are divided into four main categories, each capturing a different aspect of the state.

Adjacent danger tiles

To give the agent a sense of immediate danger surrounding it, adjacent tiles are introduced. For each of the four movements directions, a binary feature indicates if the adjacent tile in that direction contains a trap or an obstacle.

Let $d \in \mathcal{D} = \{\text{up, down, left, right}\}$ denote a movement direction. The corresponding feature is defined as

$$\phi_{\text{adj},d}(s) = \begin{cases} 1, & \text{if the adjacent tile in direction } d \text{ is dangerous or impassable,} \\ 0, & \text{otherwise.} \end{cases}$$

These features provide the agent with direct information about the immediate consequences of moving in each direction.

These features encode relative rather than absolute spatial information, meaning that the same feature is activated whenever danger occurs immediately next to the agent, regardless of its position on the grid. Consequently, experience obtained from this configuration can contribute to the value estimates of other states presenting the same local condition.

Quadrant features

Adjacent features provide information about immediate danger, but do not summarize the broader structure of one particular direction. To provide the agent with a more general indication of which directions contain potentially costly regions, the local view is divided into four directional quadrants corresponding to the cardinal directions.

Rather than encoding the exact position of each visible object, the quadrant features aggregate information over multiple cells in the same direction. Each quadrant contains six tiles, as illustrated in Figure 3, which provides an aggregated measure of the amount of danger present in each direction while reducing sensitivity to the precise location of individual traps or obstacles.

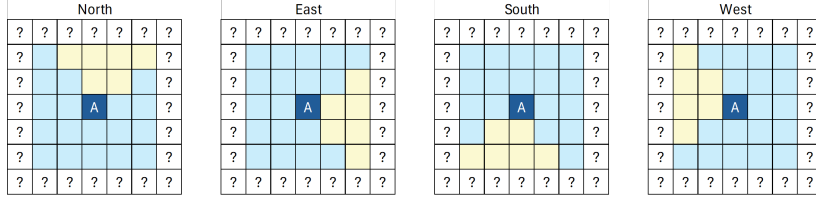


Fig. 3 Cardinal quadrants, composed by the yellow marked tiles. The "?" marked tiles are inside fog of war, hence not visible to the agent.

The same set of six features is computed independently for each of the four quadrants, resulting in a total of $4 \times 6 = 24$ quadrant features. These features capture different properties of the visible region:

Trap and obstacle density. Represents the proportion of tiles within a quadrant that contain a visible trap. For a quadrant q , it is defined as

$$\phi_{\text{trap_density},q}(s) = \frac{N_{\text{trap},q}}{N_q},$$

where $N_{\text{trap},q}$ is the number of identified traps in quadrant q , and N_q is the total number of tiles belonging to that quadrant, which is always six, hence $N_q = 6$. For example, if three out of six tiles in the north quadrant contain traps, the corresponding trap density is $3/6 = 0.5$.

Obstacle density is defined analogously and represents the proportion of cells within a quadrant occupied by obstacles:

$$\phi_{\text{obstacle_density},q}(s) = \frac{N_{\text{obstacle},q}}{N_q},$$

Trap and obstacle proximity index. While trap density measures the number of traps within a quadrant, it does not assess how close these dangers are to the agent. To incorporate this notion, a trap proximity index is introduced.

For each visible trap i in quadrant q , its "danger" contribution is determined from its Manhattan distance d_i to the agent:

$$c_i = \exp[-\lambda(d_i - 1)],$$

where $\lambda > 0$ controls the rate at which the contribution decreases with distance (in this implementation, $\lambda = 1$). Consequently, an adjacent trap to the agent with $d_i = 1$ produces the maximum contribution $c_i = 1$, while more distant traps contribute progressively smaller values.

When multiple traps are present within the same quadrant, their individual contributions are aggregated as

$$\phi_{\text{trap_proximity},q}(s) = 1 - \prod_{i=1}^{N_{\text{trap},q}} (1 - c_i).$$

This aggregation outputs a value between $[0, 1]$ and increases when traps are closer to the agent. For example, the presence of a unique adjacent trap produces a proximity value of 1, whereas multiple distant traps result in a smaller combined value.

The obstacle proximity index is defined in the same manner as the trap proximity index, replacing the set of traps with the set of visible obstacles.

$$\phi_{\text{obstacle_proximity},q}(s) = 1 - \prod_{i=1}^{N_{\text{obstacle},q}} (1 - c_i).$$

Clear pathing. Indicates whether a traversable route path exists from the agent’s current position to the outer boundary of its visible region. A path is considered clear if it can be traversed without crossing an obstacle, traps are considered traversable although they incur in a negative reward when stepped on.

$$\phi_{\text{clear},q}(s) = \begin{cases} 1, & \text{if a traversable path exists through quadrant } q, \\ 0, & \text{otherwise.} \end{cases}$$

Clear pathing provides direct information about whether the agent can reach new unexplored regions from the current state. The same algorithm for checking if a map configuration has a viable path from start to goal is used here to determine if there is a clear path outside the quadrant.

Boundary obstruction. It characterizes how strongly a quadrant provides access to unexplored regions of the map. It is similar to clear pathing in the sense that both take into consideration if a general direction can be traversed. The difference is that boundary obstruction measures how wide is the edge of each quadrant to traverse.

Let $\mathcal{B}_q$ denote the set of boundary tiles associated with quadrant q . Boundary obstruction measures the proportion of these tiles being other than obstacles (e.g. free movement tiles):

$$\phi_{\text{free},q}(s) = \frac{N_{\text{open},q}}{|\mathcal{B}_q|},$$

where $N_{\text{open},q}$ denotes the number of boundary cells classified as traversable.

If there are many boundary tiles that are obstacles, then the value of the feature would be lower, meaning that the path outside the current local view is tighter.

Exact tile view

While previously introduced features provide a general picture about the agent’s surroundings, by definition they discard specific positional information. Different configurations of traps and obstacles within the same quadrant may, in theory, produce identical density and proximity values. After experimentation, an additional set of features explicitly encodes the contents of the entire local view.

For each position p in the local view, a separate binary feature is defined for every possible tile type. Let

$$\mathcal{T} = \{\text{fog, empty, obstacle, trap}\}$$

denote the set of possible tile types. The corresponding feature is defined as

$$\phi_{p,\tau}(s) = \begin{cases} 1, & \text{if the tile at position } p \text{ is of type } \tau, \\ 0, & \text{otherwise,} \end{cases}$$

where $\tau \in \mathcal{T}$.

Consequently, each position is represented by a one-hot encoding of its tile type. Since the local view contains 25 positions and four possible tile types are considered, this representation contributes

$$25 \times 4 = 100$$

features to the complete feature vector.

It is evident that this feature set retains more detailed positional information, and proved to be a positive addition to the model as it will be covered in the results section.

Goal features

In addition to danger and traversability, the feature representation provides the agent with information about the goal whenever it is visible by the agent within the local view. These features allow the estimated action values to depend not only on the presence of the goal, but also on its relative position with respect to the agent.

The first feature, *goal in view*, is a binary indicator defined as

$$\phi_{\text{goal_visible}}(s) = \begin{cases} 1, & \text{if the goal is contained within the local view,} \\ 0, & \text{otherwise.} \end{cases}$$

When the goal is visible, two additional features encode its relative row and column positions within the local view. Denoting the relative to the agent goal coordinates by r_g and c_g , the corresponding features are

$$\phi_{\text{goal_row}}(s) = r_g, \quad \phi_{\text{goal_col}}(s) = c_g.$$

Together, these features indicate both whether the goal is currently observable and the direction in which it is positioned relative to the agent.

Bias term

Finally, a constant bias feature is appended to the feature vector:

$$\phi_{\text{bias}}(s) = 1.$$

4 Experimental results

This section presents the experimental results of the proposed learning methods within their respective environment, which are organized progressively with each setting introducing an additional level of complexity to better contrast the generalization capabilities for each case.

All agents are trained for 5000 episodes. Performance is evaluated every 100 training episodes using an evaluation batch of 100 episodes. Depending on the implementation, evaluation maps can be fixed or randomized. The learned policy is assessed independently of the exploratory behavior used during training.

Furthermore, the learning rate α and discount factor γ are selected separately for each model, since the appropriate hyperparameters may vary with the complexity of the state representation and learning method.

To reduce the influence of variation, each experimental configuration is repeated using 10 independent random seeds. The reported metrics correspond to averages across these runs, providing a more robust estimate of model performance than the results of a single training run.

4.1 General overview

4.1.1 Baseline

We first evaluate tabular Q-learning on a fixed map layout, with the objective of establishing a baseline before introducing other aspects that are proper to the original problem definition. In this sense, the environment used for this experiment is the same as Figure 1, where positions of traps and obstacles remain unchanged throughout training and evaluation.

Performance

Under this setting, the agent achieves strong performance. The average return over the last 50 training episodes is 76.8, while the corresponding mean episode length is 22 steps. These results indicate that the learned policy is able to navigate the environment efficiently, largely avoiding costly interactions with traps and obstacles.

Likewise, the agent achieves a 100% success rate on the same fixed map during evaluation. Since both training and evaluation are performed on an identical environment configuration, this result shows that tabular Q-learning is able to learn a reliable policy for this fixed configuration. However, this experiment is not enough to assess the agent’s ability to generalize to previously unseen map configurations.

The resulting learning curve confirms that the algorithm converges and the heatmap shows that the agent has a defined behavior from beginning to end, which is to move south-east in direction of the goal region.

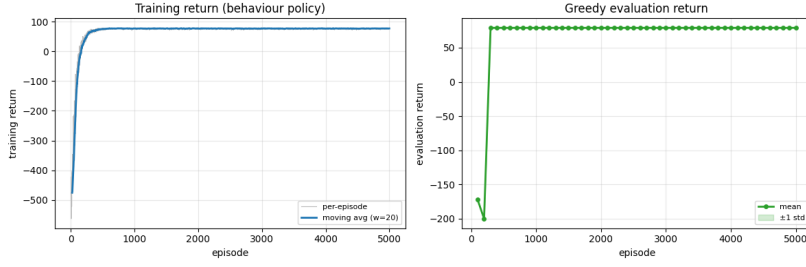


Fig. 4 Train and evaluation total return for baseline tabular Q-learning

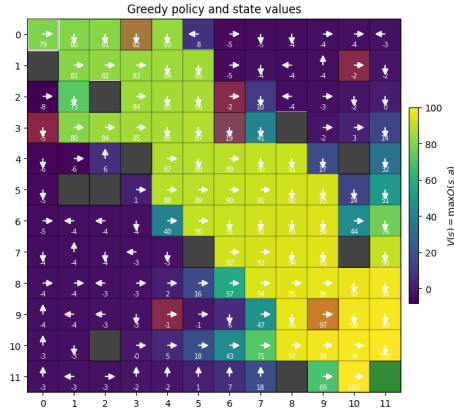


Fig. 5 Heatmap for the tabular Q-learning implementation. Black squares correspond to obstacles and red squares to traps.

Although the fixed map layout produces strong results, our main interest lies in evaluating how well the agent can adapt to variations in the environment. The previous experiment only demonstrates that tabular Q-learning can learn an effective policy for a single, known map configuration. If the resulting policy was evaluated on a different map, its performance would be expected to decrease substantially, since the learned Q-values are tied directly to the states encountered in the original map.

4.1.2 Random-layout tabular Q-learning

To recall the original problem definition, we need to consider a setting in which a new randomized map is generated at the beginning of every training episode. The positions of traps and obstacles consequently vary throughout training, requiring the agent to learn under a distribution of different environment configurations.

This change also requires careful consideration of the evaluation procedure. Generating a new set of random maps at every evaluation point and report the metrics would introduce an additional source of variability into the evaluation, since we couldn't effectively compare models within the same evaluation set.

In theory, some evaluation layouts could contain configurations that are inherently easier or more difficult than others. Consequently, differences between agents could be partially explained by differences in the sampled maps rather than by changes in the quality of the learned policy. We instead construct a fixed evaluation set for a more controlled evaluation, where a specific random seed is used to generate a predetermined collection of randomized maps, which is kept constant throughout all following experiments. Every 100 training episodes, the current policy is evaluated on this same collection of maps.

Performance

Under this randomized layout setting, performance of tabular Q-learning decreases substantially. For the last 50 episodes, the average return is -154 which indicates that the agent accumulates considerably more penalties while navigating these randomized layouts, either through stepping more often on traps or obstacles, choosing inefficient trajectories or failure to reach the goal. The agent achieves an average success rate of only 9.6% in the evaluation set, meaning that the policy learned during training transfers poorly across different map layouts.

Both metrics suggest that the state representation used by standard tabular Q-learning is insufficient to capture the information required to select appropriate actions within this setting. This limitation is natural for tabular representation, since Q-values remain indexed only by the agent’s state and action. The same state may correspond to very different surroundings depending on the map configuration, hence, an action that is advantageous in one layout may not be the same in another. Therefore, since the agent doesn’t have any other type of information to distinguish between situations, it’s forced to learn with what it is given.

The learning curves averaged across different training seeds further illustrate this limitation: training has substantial variability and a slight negative trend which is different from the stabilized curve of the fixed map setting covered previously.

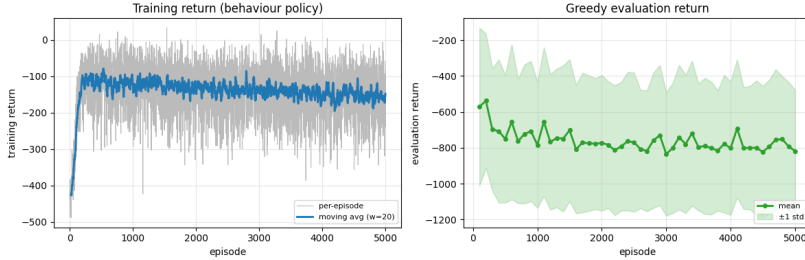


Fig. 6 Train and evaluation total return for tabular Q-learning for the randomized training setting

4.1.3 Indexed Q-Learning

Since the agent is able to observe its immediate surroundings, it can be incorporated into the state representation. As defined previously, the agent’s local view consists of direct information on what happens in its vicinity of two tiles from its current position, being a total 25 tiles.

To preserve the tabular nature of learning algorithms and contrast what would be the result of increasing the state space, an extended implementation using an indexed dictionary is presented. While standard Q-learning associates a Q-value with a state-action pair, this implementation incorporates the flattened local view also inside this “index” or “key”. The corresponding dictionary key can be expressed as

$$i = (s, a, \text{flatten}(\ell)),$$

where s represents the agent’s current position, a is the selected action, and ℓ denotes the 5×5 local observation. Each unique key is a combination of position, action and local view, and it functions exactly the same as a tabular key from the first baseline method.

Performance

Despite providing the agent with more information about its surroundings, this representation does not result in improved performance when executed for the randomized training setting. Moreover, both the training and evaluation returns are lower than those obtained with the simpler tabular Q-learning agent trained on randomized maps. The return of the last 50 training episodes decreases to -302 , while the success rate for both training and evaluation remains at 0.2% and 14.3%, respectively.

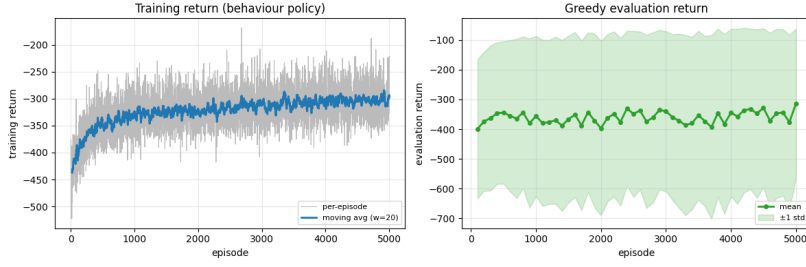


Fig. 7 Train and evaluation total return for indexed Q-learning for the randomized training setting

The poor performance can be explained by the significant increase in the state space. Every distinct local view produces a different dictionary key, even when two configurations differ by only a single tile. In this sense, experience acquired for one local configuration cannot be directly transferred to another similar configuration, and since randomized maps can generate a very large number of distinct local observations, many indexed states are encountered only rarely, greatly limiting the number of updates that can be performed for each individual Q-value.

4.2 Linear Function Approximation (LFA)

The results from tabular Q-learning motivate a different approach to the problem. This is where we introduce value function approximation, where the proposed new implementation uses linear function approximation as described in the previous section. Instead of requiring the agent to encounter the exact same state representation multiple times, states that share similar features also share parameters in the value function.

Feature representation also provides greater control over the information available to the learning algorithm. In particular, the feature engineering process determines both which properties of the environment are emphasized and the dimensionality of the representation.

In this setting, the algorithm combines function approximation with bootstrapping and off-policy learning, forming the three components commonly referred to as the *deadly triad* [?]. Unlike the tabular case, convergence is therefore not generally guaranteed, which will be also tested shortly.

The first tested LFA configuration uses adjacent danger, quadrant and goal features (as described in the feature engineering section), which by itself had a noticeable performance increase in comparison to the previous tabular model. Further inspection of individual trajectories would reveal that in some cases, the learned policy would make the agent to loop around a small set of circular map positions.

The most likely explanation for this behavior is the limited amount of information the agent has that makes it progress towards the goal. Although goal features were created for this purpose, they are only truly useful when the goal falls within the agent’s local view. Outside this region, the agent does not possess any information indicating which direction will lead the goal (e.g. ”breadcrumbs”). Previously it relied on its position and memorizing where the goal was, but here its actions only depend on its local view. Hence, in view of reducing total penalty, the agent decides to loop through low penalty tiles until the episode ends.

Another limitation could be the aggregated nature of several features, which in theory can produce identical or very similar feature vectors. Although sharing similar states is one of the advantages of VFA, extensive aggregation can remove distinctions that are relevant for selecting an action.

In view of this situation, second feature configuration is introduced that incorporates the agent’s exact tile view within its observable vicinity. As previously described, each observable tile and its corresponding type are explicitly represented within the feature vector. This representation complements the aggregate features rather than replacing them. Following results will correspond to this final LFA configuration.

Hyperparameter selection

In addition to refining the feature representation, a series of experiments were performed to identify suitable learning hyperparameters for the LFA method. This was particularly relevant for the learning rate α and discount factor γ .

As a note, tuning was not emphasized for the previous indexed tabular configuration because its primary limitation was representational. Changing the learning parameters alone could not directly address this limitation of the curse of dimensionality. In contrast, the behavior of the LFA agent is more sensitive to parameter selection, since every update modifies shared weights that may affect value estimates across multiple states.

Among the configurations evaluated, the most consistent performance was obtained using an initial learning rate of $\alpha = 0.0075$ and a discount factor of $\gamma = 0.97$.

Performance

The final LFA configuration produces an improvement over the previous randomized map methods. Unlike the indexed tabular agent, whose training performance remained highly unstable, the LFA agent exhibits an improvement as training progresses, with some declining episodes. By the end of training, the average training success rate across the 10 random seeds reaches approximately 71%.

Evaluation performance shows greater variability across seeds. For 7 of the 10 experimental seeds, the learned policies achieve an average evaluation success rate of 76%. The remaining 3 seeds exhibit substantially poorer evaluation performance, although they show evidence of learning during training. A similar improvement can be observed in the return, achieving an average return over the last 50 episodes of -44.9 . This represents a substantial increase relative to the indexed tabular approach and is consistent with the higher success rates observed during both training and evaluation.

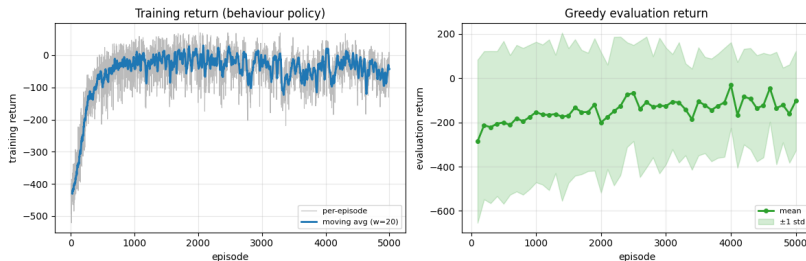


Fig. 8 Train and evaluation total return for LFA Q-learning

Overall, this demonstrates that the resulting reduction in representational sparsity allows experience collected from one configuration to contribute to action-value estimates in similar configurations. Nevertheless, the variation observed across random seeds indicates that stable learning is not achieved in every run, which remains consistent with the described difficulties associated with combining off-policy learning, bootstrapping, and function approximation.

Finally, another aspect is that compared to previous implementations, LFA takes considerable more time to execute, being the latter six times more time consuming than indexed Q-learning.

4.3 Larger map size

To evaluate how these different approaches scale as the environment increases in size, the same experiments were repeated on a 24×24 grid. The experimental settings and evaluation methodology is essentially the same as the 12×12 experiments. However, to account for the larger environment, the number of training episodes was increased from 5000 to 7500, while the maximum episode length was increased from 200 to 400 steps.

The results of these experiments are summarized in Table 2, excluding indexed Q-learning implementation since, by its nature and what was discovered, it is most likely to behave poorly considering that the state space increases even more in this case. Overall, the tabular approaches exhibit behavior consistent with that observed in the smaller environment. However, the LFA implementation does not reproduce the same improvements obtained for the 12×12 configuration and instead exhibits poorer training and evaluation performance.

One possible explanation for this result is the increased sensitivity of the LFA agent to its hyperparameters. As observed in the original experiments, obtaining stable learning behavior required tuning parameters such as the learning rate α and discount factor γ . Hence, increasing the map size changes

Table 2 Performance metrics for large map layouts (24x24).

Method	Final ϵ	Return (last 50)	Eval. return	Eval. return std.	Weight shape	Eval. success (%)	Mean ep. length	Train success (%)
Fixed tabular Q-learning	0.05	50.17	55.00	0.00	N/A	100.0	46.00	97.31
Random tabular Q-learning	0.05	-354.41	-808.17	334.36	N/A	0.10	199.85	83.92
LFA	0.05	-407.88	-208.79	82.78	-4.132	0.00	200.00	7.24

the characteristics of the navigation problem, including the typical trajectory and the amount of experience required to reach the goal. Consequently, the hyperparameters identified for the smaller environment cannot necessarily be expected to remain valid for this larger setting.

In this sense, some additional hyperparameters were evaluated, but since the computational cost of the larger environment increased with its size, it was not feasible to test a more extensive tuning space. Therefore, the poor LFA performance observed in this experiment is not conclusive evidence that linear function approximation is inherently unable to generalize to larger maps. In other words, the experiment demonstrates that the configuration that performed successfully on the smaller environment does not directly transfer to the larger setting.

Even more important are the considerations discussed with regard to the deadly triad, since we combine function approximation, bootstrapping and off-policy learning, convergence is not guaranteed.

A more extensive hyperparameter search for larger settings is therefore left as a limitation of the project and an important direction for future work. Such an analysis would be necessary to determine whether the poor performance observed in the 24×24 environment is primarily caused by lacking hyperparameter selection, increased learning instability, limitations of the current feature representation, or a combination of all of these.

5 Concluding Remarks

This project explored the ability of different Q-learning representations to learn and generalize within a partially observable grid navigation environment. The experiments progressively increased in difficulty, beginning with standard tabular Q-learning on a fixed map, followed by randomized map configurations, an indexed tabular representation incorporating the agent’s local view, and finally Q-learning with linear function approximation.

For tabular Q-learning, keeping the environment remained unchanged, the agent learned a reliable policy and achieved a 100% evaluation success rate. However, this result depended strongly on the stationarity of the map, and once traps and obstacles were randomized between episodes, position based state representation became insufficient, since same grid positions could correspond to substantially different local conditions across different maps.

Continuing with the tabular representation plus complete local observation addressed this ambiguity but introduced a different constraint. Each distinct combination of position and local view became an independent entry in the Q-value dictionary, substantially increasing the state space. Consequently, many states were encountered only rarely and experience could not be transferred between similar local configurations. The resulting indexed Q-learning agent therefore performed worse than the simpler tabular representation even when having access to more information about the environment.

Linear function approximation provided an effective result between these two representations. By using a feature set, experience obtained in one state could influence value estimates in other states with similar characteristics. Feature engineering proved to be an important component of this process. Initial aggregate features describing danger, traversability, and the relative position of the goal provided useful information but were not always sufficient to distinguish between local configurations. Incorporating the exact local tile representation included additional spatial information while maintaining the parameter sharing offered by function approximation.

The final LFA configuration produced a substantial improvement over the randomized tabular approaches. Across the 10 training seeds, the final training success rate reached approximately 71%. Furthermore, for 7 of the 10 seeds, the learned policies achieved an average evaluation success rate of approximately 76%. The average return over the final 50 training episodes also improved to -44.9 , which is favorable compared to the substantially lower returns obtained using the indexed tabular representation. These results provide evidence that linear function approximation offers a considerably more effective mechanism for generalization across randomized map configurations than the evaluated tabular alternatives.

While results overall were an improvement, the LFA agent did not achieve truly stable behavior across every random seed, with three runs exhibiting poor evaluation performance. This sensitivity is particularly relevant because the learning algorithm combines function approximation, bootstrapping, and off-policy learning, for which general convergence is not guaranteed. Additionally, this implementation introduces more computational cost, requiring considerably more execution time than the indexed tabular approach.

In general, the experiments illustrate that increasing the amount of information available to an agent is not by itself an improvement to learning, and it relies heavily on how it is represented. The indexed tabular experiment proved to be an example of this situation: a richer state representation can actually make learning more difficult when each observation must be learned independently. In contrast, function approximation allows information to be shared across related observations, making the choice of representation and features a central component of successful generalization.

Future work could investigate more expressive function approximators, especially for the larger map configurations, improved feature representations, and alternative reinforcement learning algorithms with greater training stability. Particularly, a learning strategy where difficulty is increased at a specific number of episodes would be a favorable addition and would increase efficiency in learning. The experiments could also be extended to even larger maps and different environment distributions in order to determine how the observed approximate methods scale as the state and if the features for one smaller layout worked at a larger scale, which in this case was not the case at first.

References

- [1] Ferrara, A.: Lecture notes. Università degli Studi di Milano. Reinforcement Learning (2026)
- [2] Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. The MIT Press, Cambridge, Massachusetts (2018)
- [3] Géron, A.: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd edn. O'Reilly Media, Sebastopol, California (2022)